

Project Work Plan and Progress Report

Uncertainty-Aware Active Vision for UAV-Based Infrastructure Inspection Using Deep Reinforcement Learning

Submitted By

Team : 08
Names : Kommu Shiva Gnana Yeseswini, Nunna Leekhith Sri Krishna,
Mukkheshh V
Register Numbers : BL.EN.U4AIE23013, BL.EN.U4AIE23060, BL.EN.U4AIE23036
Programme : B.Tech in Computer Science and Engineering (Artificial Intelligence)
Course : Reinforcement Learning
Institution : Amrita Vishwa Vidyapeetham, Bengaluru Campus
Submitted To : Dr. Amudha J.
Date : 28 September 2026

1 Project Problem and Work Direction

1.1 Problem We Are Solving

Unmanned Aerial Vehicles (UAVs) are increasingly used to inspect civil infrastructure such as bridges, power lines and railway structures. Current UAV inspection pipelines are good at *detecting*, classifying and segmenting defects such as cracks, but they do not decide *where to look next*. The UAV typically follows a predefined raster (lawn-mower) path and processes every region uniformly. This causes long inspection times on a limited battery, wasted computation on regions that are unlikely to contain defects, and missed or poorly segmented defects, because a fixed path cannot return for a better view of an uncertain region.

Inspection is therefore a sequential decision-making problem: each viewpoint the UAV chooses changes what it knows about the structure, and that knowledge should guide the next viewpoint. The central question of the project is:

How can a UAV learn an inspection policy that uses its own detection uncertainty to choose informative viewpoints, so that it inspects the structure fully in less time and with less energy than passive scanning?

1.2 Motive

The motivation is to turn the UAV from a passive image collector into an *active* perceiver. Instead of spending equal effort everywhere, the agent should spend effort where the detector is unsure and move on quickly where it is confident. Reinforcement learning (RL) is a natural fit because the problem involves interaction, sequential decisions, delayed feedback (the value of a flight path is known only when the inspection is complete) and no labelled “correct path”.

1.3 Project Work Direction

The work is organised as a hierarchical, multi-agent RL system in which each team member owns one agent with its own Markov Decision Process (MDP), algorithm and exploration strategy:

- **Part I – RL formulation and implementation (Assessment 1):** build a shared simulation environment, formulate three cooperating agents, implement them with Deep RL, and compare them with scripted baselines.
- **Part II – Analysis and extension:** convergence, sample efficiency, regret and exploration–exploitation analysis (CO4/CO5), integration of all three agents, and transfer from a simulated detector to a real-image detector trained on crack images.

1.4 Current Work Status

Table 1 summarises the state of each component on the date of this report.

Table 1: Component status.

Component	Owner	Status
Shared Gymnasium environment (4 modes)	Yeseswini	Complete and tested; baselines evaluated
Agent A – viewpoint planner (DQN)	Yeseswini	Trained; beats dense raster scan in time and energy
Agent B – view refiner (PPO)	Leekhith	Implemented and tested; final training and analysis in progress
Agent C – mission supervisor (A2C)	Mukhesh	Implemented and tested; final training and analysis in progress
Real-image detector (DeepCrack U-Net)	Mukhesh	Pipeline implemented and verified on a reduced run; full GPU run in progress
Integration of all three agents	Leekhith	Implemented and verified with preliminary models
Part II analysis (CO4/CO5)	All	In progress

2 Simulation Environment

2.1 Choice of Environment

Photorealistic simulators such as AirSim or Gazebo require substantial engine and middleware setup and produce slow episodes. Since the assessment focuses on the RL formulation, a lightweight custom environment was built using the Gymnasium interface [10], which allows standard algorithm implementations from Stable-Baselines3 [9] to be used directly. All training runs on Google Colab.

2.2 Inspection Setting

- The bridge surface is an 8×16 grid of cells; 20% of cells contain a hidden defect, re-sampled every episode with a fixed seed for reproducible evaluation.
- The UAV flies at one of three altitudes. High, mid and low altitude give camera footprints of 5×5 , 3×3 and 1×1 cells respectively; lower altitude gives a sharper view.
- Five view-quality levels $q = 0, \dots, 4$ are defined. Altitude reaches at most $q = 3$; the best view $q = 4$ is reached only through close-range refinement by Agent B.
- Each cell keeps its best view so far (*best-view fusion*); a better view never increases uncertainty.
- Battery starts at 1.0 and is consumed by moves (0.005), altitude changes (0.01) and refinement steps (0.008).

2.3 Detector Model

The environment supports two interchangeable detector modes behind the same interface:

- **Analytic mode:** per-cell uncertainty, detection probability and segmentation quality depend on view quality, with defective cells being more ambiguous than clean ones.
- **Real-image mode:** a U-Net [7] with dropout is trained on the training split of the DeepCrack dataset [8]. 64×64 patches from the *test* split only (no data leakage) are viewed at five effective resolutions. Uncertainty is the predictive entropy of the Monte-Carlo dropout mean

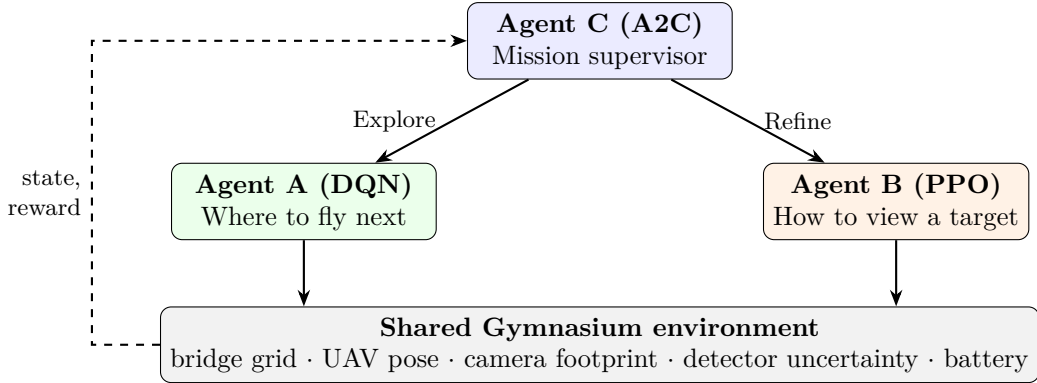
prediction [6]; detection and IoU are computed against the ground-truth masks. Results are cached so that environment steps remain fast.

2.4 Environment Interaction

Because the defect map is hidden and the UAV only sees what its camera captures, inspection is a Partially Observable MDP. In practice it is solved as an MDP over an observation-based state:

$$S_t \xrightarrow{A_t} S_{t+1}, \quad r_t = R(S_t, A_t, S_{t+1}). \quad (1)$$

The environment exposes four modes so that each agent can be trained independently against scripted stand-ins for the other two, and then combined:



The overall interaction in one mission is:

1. Initialise a bridge with a hidden defect map; the UAV starts at the base at high altitude.
2. Observe the current state (coverage, uncertainty map, pose, battery).
3. Agent C selects an option: *Explore*, *Refine* or *Return home*.
4. *Explore* runs Agent A for several steps; *Refine* runs Agent B on the most uncertain cell in view.
5. The camera observes the footprint; the detector updates per-cell uncertainty and detections.
6. Rewards are computed and the policies are updated during training.
7. The episode ends on successful inspection, return home, empty battery or step limit.

3 RL Formulation of the Three Agents

Table 2: MDP formulation of each agent.

	Agent A – Viewpoint Planner	Agent B – View Refiner	Agent C – Mission Supervisor
Decision	Where to fly and at which altitude	How to position the camera on one uncertain cell	Explore, refine or return home
State	Egocentric 7×7 windows of coverage and uncertainty, 2×4 block summary, row, column, altitude, battery (118 values)	Distance, viewing angle, lateral offset, target uncertainty, battery, time left (6 values)	Battery, coverage, mean uncertainty, fraction of defects found, distance to base, time used (6 values)
Action	Discrete(6): N, S, E, W, climb, descend	Continuous, 3-D: change in distance, angle, lateral offset	Discrete(3) options (semi-MDP)
Reward	$0.2 \Delta U - 0.01$ per step -0.05 revisit -1 out of bounds; success $+10 + 20 b_t$	$\Delta u - 0.01 \ a\ + 0.5(\gamma s' - s)$; $+5$ best view; -2 timeout	$+2$ per new defect $-0.5 \times$ energy; -10 empty battery
Algorithm	DQN [1]	PPO [2]	A2C [3]
Exploration	ϵ -greedy, $1.0 \rightarrow 0.05$	Gaussian policy σ + entropy bonus	Softmax policy + entropy bonus
Baseline	Raster scan (mid / low altitude)	Fixed orbit, random	Rule-based supervisor

Here ΔU is the total uncertainty removed in a step (unseen cells count as 1), b_t the remaining battery and s the view quality. The shaping term of Agent B is potential-based [5] and does not change the optimal policy. Agent C’s options follow the semi-MDP framework of Sutton et al. [4]. The core update rules are:

$$\text{DQN: } L(\theta) = (r + \gamma \max_{a'} Q_{\bar{\theta}}(s', a') - Q_{\theta}(s, a))^2, \quad (2)$$

$$\text{PPO: } L^{\text{CLIP}}(\theta) = \mathbb{E}_t[\min(\rho_t(\theta)\hat{A}_t, \text{clip}(\rho_t(\theta), 1 - \varepsilon, 1 + \varepsilon)\hat{A}_t)], \quad \rho_t = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}, \quad (3)$$

$$\text{A2C: } \nabla_{\theta} J = \mathbb{E}[\nabla_{\theta} \log \pi_{\theta}(a | s) (r + \gamma V(s') - V(s))] + \beta \nabla_{\theta} \mathcal{H}(\pi_{\theta}(\cdot | s)). \quad (4)$$

4 Novelty, Results and Proposed Improvements

4.1 Novelty of the Work

- **Uncertainty-driven viewpoint selection:** the agent’s state is built from the detector’s own uncertainty, so the policy decides where more evidence is needed instead of scanning uniformly.
- **Hierarchical decomposition:** navigation, view refinement and mission management are separate agents with different action types (discrete, continuous, options), trained independently in one shared environment and then combined.
- **Real-image uncertainty:** uncertainty in the real-image mode comes from an MC-dropout U-Net trained on crack images, with a leakage-free split and calibrated view-quality levels.

4.2 Results So Far

Agent A (DQN) – final Part I result. Table 3 compares the trained agent with two raster scans on the same 20 seeded bridges (analytic detector, 2×10^5 training steps, 10.2 minutes on a Colab CPU).

Table 3: Agent A compared with raster baselines.

Metric	Raster (mid)	Raster (low, dense)	DQN (Agent A)
Success rate	0.00	1.00	1.00
Steps (inspection time)	199.0	123.0	87.5
Energy used	1.000	0.625	0.463
Coverage	1.000	0.953	0.998
Defects found (of 26)	20.9	23.5	22.0
mIoU	0.500	0.626	0.539
Return	6.20	35.43	35.81

The DQN matches the dense raster scan’s success rate with 29% fewer steps and 26% less energy. The trade-off is a lower mIoU: the agent inspects just enough to satisfy the uncertainty criterion, which is what the reward specifies.

Preliminary results for Agents B and C and integration. Table 4 lists short verification runs used to test the implementations. These are preliminary; final numbers will come from each owner’s full training runs.

Table 4: Preliminary verification runs (analytic detector).

Run	Result
Agent B, PPO, 6×10^4 steps	100% success in 3.7 steps; fixed orbit 0%, random 2%
Agent C, A2C, 4×10^4 decisions	Return 41.1 vs. 42.8 for the rule-based supervisor, with about half the energy (0.45 vs. 0.95)
Integration, all three learned agents	Energy 0.49 vs. 0.95 for the fully scripted system; 20.6 vs. 22.6 defects found
Real-image detector (reduced CPU run)	Uncertainty falls and IoU on defects rises from 0.23 to 0.72 as view quality improves
Agent A on real-image uncertainty	15–20% success after $1.5\text{--}2 \times 10^5$ steps; learning still improving

4.3 Lessons from the Current Work

- A global one-hot position (386 inputs) did not learn; an egocentric observation made Agent A learnable.
- A reward with separate coverage and uncertainty terms allowed reward to be collected without completing the inspection; rewarding total uncertainty reduction with a battery-scaled success bonus aligned the objective.
- With $\gamma = 0.99$ the delayed success bonus was discounted too strongly; $\gamma = 0.995$ was required.
- MC-dropout standard deviation does not rise on blurred views, because the network becomes falsely confident; predictive entropy does.
- With real-image uncertainty the reward signal is weaker and noisier, so learning needs many more samples.

4.4 Limitations Identified

- The structure is a 2D grid abstraction; wind, collisions and 3D geometry are not modelled.
- Vanilla DQN shows evaluation instability late in training; the best checkpoint is kept.

- Agent C is trained with scripted sub-agents, so plugging in learned sub-agents shifts its input distribution.
- Results are from single training seeds.

4.5 Improvement Plan (Part II)

1. Complete convergence, sample-efficiency and regret analysis for each agent (CO4).
2. Complete exploration–exploitation sensitivity studies: ϵ -decay (A), entropy coefficient and clip range (B), entropy coefficient (C) (CO5).
3. Train all agents on real-image uncertainty and compare with the analytic detector.
4. Retrain Agent C with the learned Agent A inside its *Explore* option to reduce the distribution shift.
5. Report results over multiple seeds with confidence intervals.
6. Future work: Double/Dueling DQN and a 3D simulator for sim-to-real transfer.

5 Literature-Based Direction

5.1 Deep Reinforcement Learning

Value-based methods such as DQN [1] handle discrete actions with experience replay and target networks. Policy-gradient methods such as PPO [2] handle continuous actions with stable clipped updates, and actor-critic methods such as A2C [3] suit small discrete decision spaces. The project assigns each method to the decision type it fits best.

5.2 Hierarchical RL and Options

The options framework [4] models temporally extended actions as a semi-MDP. It motivates Agent C, whose actions invoke the lower-level agents for several steps.

5.3 Uncertainty Estimation and Crack Segmentation

MC-dropout [6] provides a practical Bayesian approximation to model uncertainty. U-Net [7] is a standard architecture for pixel-level segmentation, and DeepCrack [8] provides annotated crack images used to train the real-image detector.

5.4 Reward Design

Potential-based reward shaping [5] speeds learning without changing the optimal policy and is used for Agent B.

5.5 Identified Gap

Existing inspection pipelines separate perception from flight planning: detection quality does not influence where the UAV goes next. This project closes the loop by using the detector’s uncertainty as the state for RL-based viewpoint decisions. *The team’s full literature survey and comparison table is presented in the final report.*

6 Conclusion and Next Steps

Part I has established a complete RL pipeline for uncertainty-aware UAV inspection: a shared Gymnasium environment, a simulated and a real-image detector, three agents covering discrete, continuous and option-level decisions, and an integration layer. Agent A already matches the success of dense raster scanning while using 29% fewer steps and 26% less energy. The remaining work focuses on the Part II analysis, on real-image uncertainty and on the integrated system.

6.1 Next Steps

#	Task	Owner
1	Build the real-image detector cache on GPU	Mukhesh
2	Final training and analysis of Agent B	Leekhith
3	ϵ -sensitivity, convergence and regret analysis of Agent A	Yeseswini
4	Final training and analysis of Agent C	Mukhesh
5	Agent A on real-image uncertainty	Yeseswini
6	Integration of all three agents and ablation	Leekhith
7	Report, team and individual posters, slides	All

6.2 AI Tool Acknowledgement

AI tools (Claude, Anthropic) were used to assist with planning, code development and drafting parts of this document. All design decisions, experiments and results were reviewed and verified by the team.

References

- [1] V. Mnih et al., “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, pp. 529–533, 2015.
- [2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford and O. Klimov, “Proximal policy optimization algorithms,” arXiv:1707.06347, 2017.
- [3] V. Mnih et al., “Asynchronous methods for deep reinforcement learning,” in *Proc. ICML*, 2016.
- [4] R. S. Sutton, D. Precup and S. Singh, “Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning,” *Artificial Intelligence*, vol. 112, pp. 181–211, 1999.
- [5] A. Y. Ng, D. Harada and S. Russell, “Policy invariance under reward transformations: Theory and application to reward shaping,” in *Proc. ICML*, 1999.
- [6] Y. Gal and Z. Ghahramani, “Dropout as a Bayesian approximation: Representing model uncertainty in deep learning,” in *Proc. ICML*, 2016.
- [7] O. Ronneberger, P. Fischer and T. Brox, “U-Net: Convolutional networks for biomedical image segmentation,” in *Proc. MICCAI*, 2015.
- [8] Y. Liu, J. Yao, X. Lu, R. Xie and L. Li, “DeepCrack: A deep hierarchical feature learning architecture for crack segmentation,” *Neurocomputing*, vol. 338, pp. 139–153, 2019.
- [9] A. Raffin et al., “Stable-Baselines3: Reliable reinforcement learning implementations,” *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.
- [10] M. Towers et al., “Gymnasium: A standard interface for reinforcement learning environments,” arXiv:2407.17032, 2024.